

项目维护指南

本文档面向维护者与二次开发者，说明当前模板工程的架构约定、实现方式，以及驱动、BSP、多项目扩展的正确做法与禁止行为。

架构概览见 [architecture.md](#)。原始设计目标见仓库根目录 `嵌入式统一模块化架构` 技术说明 `.md`。

1. 设计原则（维护时必须遵守）

原则	含义
核心与硬件解耦	应用层只通过 <code>devmgr_get_*</code> () 访问设备，不包含具体驱动头文件
驱动即插即用	新增驱动 = 新增 <code>.c</code> + 实现标准接口 + <code>REGISTER_DRIVER</code> ， 不改 <code>devmgr.c</code> / <code>main</code> 初始化列表
资源静态分配	禁止依赖 <code>libc</code> 堆；动态需求走 <code>mem_pool</code> （覆盖 <code>malloc/free/realloc</code> ）或编译期静态缓冲
板级与逻辑分离	引脚、外设实例、波特率写在 <code>board_config.h</code> ；驱动只读配置，不写死 GPIO
版本差异在编译期	不同成品通过 CMake 选源文件 + 项目级 <code>board_config</code> 区分，不改公共核心
HAL 统一入口	驱动/应用不直接调用 SPL 函数（除 <code>board_config.h</code> 中的类型引用），统一走 <code>*_hal_*</code>

2. 当前架构

2.1 分层与目录

```
project_template/
├── projects/<VERSION>/app/           # 应用层：业务、任务、回调、
board_config.h
├── projects/<VERSION>/<VERSION>_upper_ui_flutter/ # Flutter 主上位
机（默认 App/web）
├── projects/<VERSION>/<VERSION>_upper_ui/         # 可选 uni-app
小程序/历史兼容栈
├── common/
│   ├── interfaces/           # 标准接口
(sensor/display/actuator/comm/misc)
```

```

|   └─ driver_core/           # REGISTER_DRIVER、.driver_list 链接
段
|   └─ device_manager/        # devmgr_init_all、按名查询
|   └─ scheduler/             # 协作调度、sched_loop、事件阻塞/唤醒
|   └─ irq_event/             # 中断源 → 调度事件映射
|   └─ app_framework/         # app_fsm (ZNCZ_001 UI 导航已接入)
|   └─ utils/                 # comm_port、mem_pool、cJSON、
cjson_port、tiny_printf、soft_uart
└─ drivers/                   # 外设驱动 (按类型分子目录)
└─ bsp/
|   └─ hal_wrapper/           # GPIO/I2C/USART/TIMER/SPI HAL
|   └─ irq_handlers/          # 统一 IRQ 入口, 转发至 HAL
|   └─ board/                 # board_config 模板
|   └─ linker_scripts/        # .driver_list 段定义
|   └─ stm32f10x_std_lib/      # CMSIS + SPL
└─ cmake/                    # 工具链、hal_options.cmake、
driver_catalog.cmake
└─ tools/
    └─ gen_project.py          # 新建项目脚手架 (含上位机 scaffold 与
版本特性模板)
    └─ export_project.py       # 按版本导出嵌入式 + 上位机的独立包
    └─ cmake/                 # 导出解析脚本 (resolve_export.cmake
等)

```

2.2 控制流与数据流

```

上电
└─ bsp_init()                 NVIC 分组、DWT、SysTick(1ms)、debug_uart
(可选)
└─ sched_init()
└─ irq_event_init()
└─ irq_event_bind(...)        可选: USART RX → APP_EVENT_*
└─ devmgr_init_all()          遍历 .driver_list, 调用各驱动 init()
└─ sched_add_task(...)        注册应用任务
└─ sched_start()              永不返回, 协作调度

中断 (USART/TIM)
└─ irq_handlers.c → *_hal_irq_handler()
└─ irq_event_post_from_isr() / event_set_from_isr()
└─ 唤醒 task_block() 中等待对应事件的任务

应用任务
└─ devmgr_get_*("name")       获取接口指针
└─ comm_port_* / esp8266_mqtt_* 通讯与 MQTT (若启用)
└─ app_logic_loop / sched_loop 业务逻辑与周期 poll
└─ 调用接口方法 (read/send/print 等)
└─ task_block(events)         让出 CPU, 等待 tick/传感器/通信事件

```

2.3 驱动自注册机制

每个驱动在文件底部注册：

```
static const temp_hum_sensor_t dht11_drv = {
    "dht11",          /* 全局唯一名称，供 devmgr 查询 */
    dht11_init,
    dht11_read_temperature,
    dht11_read_humidity
};

REGISTER_DRIVER(TEMP_HUM_SENSOR, dht11_drv);
```

链接脚本保留 `.driver_list` 段，符号 `__driver_list_start` / `__driver_list_end` 供 `devmgr.c` 遍历。

当前支持的驱动类型 (`driver_core.h`)：

REGISTER 类型	接口结构体	devmgr 查询 API
TEMP_HUM_SENSOR	temp_hum_sensor_t	devmgr_get_temp_hum_sensor()
DISPLAY	display_driver_t	devmgr_get_display()
SERVO	servo_driver_t	devmgr_get_servo()
RELAY	relay_driver_t	devmgr_get_relay()
COMM	comm_driver_t	devmgr_get_comm()
MISC	misc_driver_t	devmgr_get_misc()
DISTANCE_SENSOR	distance_sensor_t	devmgr_get_distance_sensor()
WEIGHT_SENSOR	weight_sensor_t	devmgr_get_weight_sensor()
GAS_SENSOR	gas_sensor_t	devmgr_get_gas_sensor()
ANALOG_PROBE	analog_probe_t	devmgr_get_analog_probe()
PRESSURE_SENSOR	pressure_sensor_t	devmgr_get_pressure_sensor()
IMU_SENSOR	imu_sensor_t	devmgr_get_imu_sensor()
PULSE_OXIMETER	pulse_oximeter_t	devmgr_get_pulse_oximeter()
BLOOD_PRESSURE	blood_pressure_sensor_t	devmgr_get_blood_pressure()
RTC	rtc_driver_t	devmgr_get_rtc()
INPUT	input_driver_t	devmgr_get_input()

REGISTER 类型	接口结构体	devmgr 查询 API
STEPPER	stepper_driver_t	devmgr_get_stepper()
SEGMENT_DISPLAY	segment_display_t	devmgr_get_segment_display()
RFID	rfid_driver_t	devmgr_get_rfid()
RADIO	radio_driver_t	devmgr_get_radio()
GNSS	gnss_driver_t	devmgr_get_gnss()

2.3.2 key_service (中断按键)

- 默认 `drivers/misc/key.c` 负责配套 `board_key*_pin`，以 EXTI 中断 + 状态机输出单击/双击/长按。
- 应用层只包含 `common/interfaces/key_service.h`，在 `devmgr_init_all()` 之后 `key_register_callback(app_key_event_handler, ctx);`，回调中自行设置 `APP_EVENT_KEY` 并转发到业务逻辑。
- 按键驱动内部通过 `bsp/hal_wrapper/gpio_hal.c` + `exti_hal.c` 配置 GPIO/EXTI，并在 `bsp/irq_handlers/irq_handlers.c` → `irq_event` → 调度任务的链路中运行，不允许在驱动里声明 NVIC ISR。
- 如需更换引脚/数量，可在 `board_config.h` 重定义 `KEY_DRIVER_PIN_TABLE` / `KEY_DRIVER_BUTTON_COUNT`。

2.3.1 已迁移驱动清单

驱动源文件统一在 `drivers/`，CMake 选用见 `cmake/driver_catalog.cmake`：

注册名	源文件	接口
dht11 / ds18b20	sensors/dht11.c, ds18b20.c	temp_hum_sensor
bmp180	sensors/bmp180.c	pressure_sensor
mpu6050	sensors/mpu6050.c	imu_sensor
max30102	sensors/max30102.c	pulse_oximeter
mcp20_bp	sensors/mcp20_bp.c	blood_pressure (RTJK V8+, 需 HAL_ADC_ENABLE)
pm25 / co2	sensors/pm25.c, co2_uart.c	gas_sensor
ph_sensor	sensors/ph_sensor.c	analog_probe

注册名	源文件	接口
hcsr04 / hx711 / ds1302	sensors/hcsr04.c, hx711.c, ds1302_rtc.c	distance / weight / rtc
rc522	sensors/rfid_rc522.c	rfid (需 HAL_SPI_USE_HW)
sg90 / sg90_2	actuators/sg90.c	servo
stepmotor	actuators/stepmotor.c	stepper
esp8266 / jdy31 / su03t / l76k / nrf24	comm/*.c	comm / gnss; esp8266_mqtt 为协议层 (不单独注册)
oled / hc595_1	displays/*.c + oled_font.c	display / segment_display
led / buzzer / key	misc/*.c	misc / input
gl5506 / e18_presence	sensors/gl5506_light.c, e18_presence.c	analog_probe
light1-3 / key_4ch	actuators/light_channel.c, misc/key_4ch.c	relay / input

全量链接验证：`projects/driver_all_test/` (26 条 `.driver_list` 注册, Flash ~19.7 KB / RAM ~3.3 KB) 。

完成度与构建指标见 [project_status.md](#); 架构分层见 [architecture.md](#)。

2.4 应用启动顺序（固定）

`app_main.c` 必须按以下顺序初始化，不可打乱：

```

void app_main(void)
{
    bsp_init();           /* 1. 板级基础: NVIC、SysTick、DWT */
    sched_init();         /* 2. 调度器 */
    irq_event_init();     /* 3. 中断事件桥 */
    irq_event_bind(...); /* 4. 可选: 绑定中断源到应用事件 */
    devmgr_init_all();    /* 5. 驱动 init (可能配置
GPIO/USART/I2C) */
    sched_loop_init();    /* 6. 可选但推荐: 框架 Loop */
    sched_loop_register(...); /* 7. 注册 sensor/key/comm/logic 等
loop */
    /* 8. 或 sched_add_task(...) 手写任务 */
    sched_start();        /* 9. 进入调度, 不应返回 */
}

```

2.5 comm_port 与 ESP8266 MQTT

comm_port (`common/utils/comm_port.c`) 在 `board_config.h` 的 `BOARD_COMM_DEVICE` 与 `devmgr_get_comm()` 之间做薄封装:

```

comm_port_bind(0);           /* NULL → 使用
BOARD_COMM_DEVICE */
comm_port_register_rx_callback(app_comm_rx_callback);
if (comm_port_has_irq()) {
    irq_event_bind(comm_port_irq_source(), APP_EVENT_COMM_RX);
}

```

ESP8266 MQTT (`esp8266_mqtt.c`) 在 `esp8266_wifi` 驱动之上实现 WiFi/MQTT 连接与 JSON 遥测, 不单独 `REGISTER_DRIVER`。应用层在 `devmgr_init_all()` 之后调用 `esp8266_mqtt_connect()`, 在 `comm_poll` 任务中调用 `esp8266_mqtt_poll()`。

2.6 mem_pool 与 cJSON

- 链接 `mem_pool.c` 后, `libc malloc/free/realloc` 由静态池实现 (默认 2048 B, 可在编译前改 `MEM_POOL_SIZE`)。
- `cjson_port_init()` 将 cJSON hooks 指向 `mem_pool`; 遥测 JSON 优先走 `esp8266_mqtt_publish_telemetry()` 或 `cjson_port_build_telemetry()`。
- **所有业务 JSON 字段必须用 cJSON 构建/解析**: 新增字段用 `CJSON_Add*ToObject()`, 解析字段用 `CJSON_GetObjectItemCaseSensitive()` 等 cJSON API。
- 禁止用 `sprintf` / `snprintf` / `strcat` / 手工拼接字符串构造 JSON; 这样会绕过 cJSON 转义、增加缓冲区溢出风险, 并导致字段格式不可统一。

- 显示格式化使用 `tiny_printf`，避免链接完整 `sprintf`；调试日志可使用标准 `printf`（`HAL_DEBUG_UART_ENABLE=1` 时经 PC13 软串口输出）。
- 调试日志不是业务协议输出；发送到 MQTT/BLE/串口协议层的 JSON 仍必须走 cJSON。

2.6.1 调试软串口 (soft_uart / debug_uart)

文件	职责
<code>common/utils/soft_uart.c</code>	DWT 定时 GPIO 位bang, 8N1, 仅 TX
<code>common/utils/debug_uart.c</code>	读取 <code>board_config.h</code> 引脚/波特率, 在 <code>bsp_init()</code> 中 init
<code>bsp/syscalls/syscalls.c</code>	<code>_write()</code> 转发 stdout/stderr

启用步骤:

1. CMake: `option(HAL_DEBUG_UART_ENABLE ... ON)` 或通过 `-DHAL_DEBUG_UART_ENABLE=ON`。
2. `board_config.h` 增加 `board_debug_uart_tx` 与 `BOARD_DEBUG_UART_BAUDRATE`（模板见 `board_config_template.h`）。
3. 链接 `soft_uart.c`、`debug_uart.c`（ZNCZ_001 已默认加入 `COMMON_SRCS`）。
4. 实板：USB-TTL RX ← PC13, GND 共地, **9600** 8N1。

注意:

- 与 `board_led_pin` 同脚时 LED 驱动自动禁用，避免引脚被二次配置。
- PC13 仅适合调试 TX，不要与 DS1302 / ESP8266 等业务 UART 复用。
- 软串口发送会关中断，高波特率或大量日志可能影响实时性；正式量产可 `OFF` 关闭。

2.7 HAL 功能开关与使用原则

通过 `cmake/hal_options.cmake` 控制，编译期注入 `hal_features.h`：

CMake 选项	效果
<code>HAL_I2C_USE_SOFT=ON</code>	使用 <code>i2c_hal_soft.c</code> ，不链接 <code>stm32f10x_i2c.c</code>
<code>HAL_SPI_USE_SOFT=ON</code>	编译 <code>spi_hal_soft.c</code>

CMake 选项	效果
<code>HAL_SPI_USE_HW=ON</code>	编译 <code>spi_hal_hw.c</code> + <code>stm32f10x_spi.c</code> (与软 SPI 互斥)
<code>HAL_SPI_ENABLE_DMA=ON</code>	硬件 SPI DMA TX/RX (需 <code>HAL_SPI_USE_HW</code>)
<code>HAL_USART_ENABLE_DMA=ON</code>	编译 <code>usart_hal_dma.c</code> + <code>stm32f10x_dma.c</code>
<code>HAL_ADC_ENABLE=ON</code>	ADC 轮询采集
<code>HAL_ADC_ENABLE_DMA=ON</code>	ADC DMA 连续采集
<code>HAL_DEBUG_UART_ENABLE=ON</code>	PC13 软串口 TX, <code>printf</code> 调试输出

`board_config.h` 中需与开关一致 (例如软 I2C 时 SCL/SDA 用
`GPIO_HAL_MODE_OUT_OD`; 软串口时配置 `board_debug_uart_tx`) 。

统一入口：驱动层不得直接 `#include "stm32f10x_*.h"`。GPIO/AFIO 用
`gpio_hal.*`, EXTI 用 `exti_hal.*`, NVIC/任务唤醒通过 `irq_event`, 其余
外设 (I2C/SPI/USART/ADC/TIMER) 同理。在 `hal_wrapper/` 内部才允许调用
SPL。

3. 新增驱动（标准流程）

3.1 步骤清单

1. **确定接口类型** — 在 `common/interfaces/` 中选已有接口; 若无合适类型, 先扩展接口 (见 §6) 。
2. **创建驱动文件** — 放入 `drivers/<category>/`, 如
`drivers/sensors/bme280.c`。
3. **实现接口 + 自注册** — 文件内 `static` 函数 + 接口实例 + `REGISTER_DRIVER`。
4. **添加板级配置** — 在 `projects/<VERSION>/app/board_config.h` 增加引脚/地址/速度宏。
5. **加入编译** — 在 `cmake/driver_catalog.cmake` 扩展 `DRIVER_CATALOG_*`; 工程 `CMakeLists.txt` 使用 `set(DRIVER_SRCS ${DRIVER_CATALOG_*})` (见 §12.2.5) 。
6. **应用层按名使用** — `devmgr_get_*("bme280")`, 检查返回值是否为 `NULL`。
7. **验证** — 编译通过; map 文件中可见 `.driver_list` 条目; 必要时用逻辑分析仪/串口验证协议。

3.2 驱动文件模板

```
#include "<interface>_if.h"
#include "board_config.h"
#include "driver_core.h"
/* 按需: gpio_hal.h / i2c_hal.h / uart_hal.h / timer_hal.h */

static void my_sensor_init(void)
{
    /* 1. 从 board_config 读取引脚/总线 */
    /* 2. 调用 HAL init, 检查 hal_status_t */
    /* 3. 初始化驱动私有静态缓存 */
}

static float my_sensor_read_temperature(void)
{
    /* 协议实现; 失败时返回上次有效值或约定哨兵值 */
    return 0.0f;
}

static const temp_hum_sensor_t my_sensor_drv = {
    "bme280",           /* 名称: 小写、简短、项目内唯一 */
    my_sensor_init,
    my_sensor_read_temperature,
    my_sensor_read_humidity /* 若无湿度, 可返回固定值或读寄存器 */
};

REGISTER_DRIVER(TEMP_HUM_SENSOR, my_sensor_drv);
```

3.3 命名约定

对象	约定	示例
驱动逻辑名	小写 + 数字/型号	"dht11", "oled", "esp8266"
board 宏	BOARD_<设备>_<属性>	BOARD_OLED_I2C_ADDR
board 引脚	board_<设备>_pin	board_relay_pin
驱动源文件	型号或功能	oled_ssd1306.c, esp8266_wifi.c

3.4 同类型多实例

同一接口类型可注册多个驱动（如两个温湿度传感器），名称必须不同：

```
REGISTER_DRIVER(TEMP_HUM_SENSOR, dht11_indoor_drv);    /* "dht11_in"
*/
REGISTER_DRIVER(TEMP_HUM_SENSOR, dht11_outdoor_drv);  /*
"dht11_out" */
```

应用层分别 `devmgr_get_temp_hum_sensor("dht11_in")` 查询。

3.5 通信驱动与回调

通信类驱动应实现 `register_rx_callback`，在 `recv` 或 USART 中断路径中调用回调。推荐通过 `comm_port` 统一访问：

```
comm_port_bind(0);    /* 使用 BOARD_COMM_DEVICE，如 "esp8266" */
comm_port_register_rx_callback(app_comm_rx_callback);
if (comm_port_has_irq()) {
    irq_event_bind(comm_port_irq_source(), APP_EVENT_COMM_RX);
}
```

ESP8266 MQTT 在 `comm_port_bind` 且设备名为 "esp8266" 时，由 `app_main` 调用 `esp8266_mqtt_connect()`；下行数据在 `comm_poll` 任务中通过 `esp8266_mqtt_poll()` 解析。

NRF24 等无 IRQ 的分包设备需在 `comm_poll` 任务中周期性调用 `comm_port_recv()`。

4. BSP 与板级支持扩展

4.1 职责边界

层级	负责内容	不负责内容
<code>board_config.h</code>	引脚、Remap、外设实例、波特率、I2C 地址、DMA 通道	业务逻辑、协议解析
<code>hal_wrapper/</code>	寄存器操作封装、超时、错误码、总线恢复	具体传感器命令
<code>irq_handlers.c</code>	统一 IRQ 入口，调用 HAL handler	应用业务
<code>hal_common.c</code>	<code>bsp_init</code> 、微秒计时、通用等待	设备特有初始化

4.2 修改引脚 / 外设 (同一 MCU)

只改 `projects/<VERSION>/app/board_config.h`:

```
#if HAL_I2C_USE_SOFT
static const hal_pin_t board_oled_i2c_scl = { GPIOB, GPIO_Pin_8,
GPIO_HAL_MODE_OUT_OD };
static const hal_pin_t board_oled_i2c_sda = { GPIOB, GPIO_Pin_9,
GPIO_HAL_MODE_OUT_OD };
#else
static const hal_pin_t board_oled_i2c_scl = { GPIOB, GPIO_Pin_8,
GPIO_HAL_MODE_AF_OD };
static const hal_pin_t board_oled_i2c_sda = { GPIOB, GPIO_Pin_9,
GPIO_HAL_MODE_AF_OD };
#endif
```

若 Remap 变化, 同步修改 `BOARD_*_REMAP` 并在驱动 init 中通过 HAL 配置 (`i2c_hal_config_t.remap` / `gpio_hal_apply_remap`)。

4.3 新增 HAL 模块 (如硬件 SPI、ADC)

- 1. 在 `bsp/hal_wrapper/` 添加 `xxx_hal.h` / `xxx_hal.c`。
- 2. 使用 `hal_status_t` 返回错误, 超时用 `hal_wait_flag_us()` / `hal_get_us()`。
- 3. 如需 IRQ, 在 `irq_handlers.c` 增加转发, **不在驱动里写** `void XXX_IRQHandler`。
- 4. 在 `cmake/hal_options.cmake` 增加 option (若可选编译)。
- 5. 更新 `board_config_template.h` 与 `architecture.md`。

4.4 更换 MCU / 更换 SPL 版本

- 1. 替换 `bsp/stm32f10x_std_lib/` 与 `startup/`、`linker_scripts/`。
- 2. 审查所有 `hal_wrapper` 中对寄存器/外设的假设。
- 3. 保留 `.driver_list` 链接段定义。
- 4. **不要**为此修改 `common/` 与 `drivers/` 的业务接口。

4.5 多 I2C 总线

场景	做法
同总线多设备	共用 <code>BOARD_*_I2C</code> 实例, 不同 <code>I2C 地址</code>

场景	做法
硬件 I2C1 + I2C2	各设备独立 <code>i2c_hal_init(&cfg)</code> , <code>cfg.instance</code> 不同
软 I2C 超过 2 路	需扩展 HAL: 引入 <code>bus_id</code> 、增大槽位数组 (当前软 I2C 以 I2C1/I2C2 为逻辑 ID)

5. 多项目与多版本扩展

5.1 版本模型

共享（所有版本相同）	版本专属（每个项目独立）
<code>common/*</code>	<code>projects/<VERSION>/app/</code>
<code>bsp/hal_wrapper/*</code>	├─ <code>app_main.c</code>
<code>bsp/irq_handlers/*</code>	├─ <code>app_callbacks.c</code>
<code>drivers/*</code> （源文件库）	├─ <code>board_config.h</code>
<code>cmake/*</code>	├─ <code>version_config.h</code>
	└─ <code>CMakeLists.txt</code> （选 <code>DRIVER_SRCS</code> ）

原则：公共代码不因某一版本特殊需求而分叉；版本差异通过「选哪些驱动编译」和「`board_config` 内容」表达。

统一版本宏强制要求：

- 全局版本输入宏只能使用 `APP_VERSION`，并且必须由工程 `CMakeLists.txt` 以 `CACHE STRING` 定义。
- C 代码里的功能条件输出只能使用 `VERSION_FEATURE_*`，并统一在 `projects/<NAME>/app/version_config.h` 中由 `APP_VERSION` 派生。
- 应用层、板级配置和回调代码只判断 `VERSION_FEATURE_*`；除 `version_config.h` 和工程 CMake preamble 外，不得直接用 `APP_VERSION` 写业务分支。
- 旧项目宏（如 `KQZL3_VERSION` / `RTJK_VERSION` / `KQZL3_HAS_*`）仅可作为兼容别名保留在 `CMakeLists.txt` 或 `version_config.h` 中，新代码不得新增或继续消费项目专属版本/功能宏。

5.2 新建项目步骤

```
cd project_template
python tools/gen_project.py RTJK_001
```

生成 `projects/RTJK_001/`，包含：

- `app_main.c` (最小启动骨架)
- `board_config.h` (自模板复制)
- `version_config.h` (`VERSION_NAME`、应用事件定义)
- `CMakeLists.txt` (需手动编辑 `DRIVER_SRCS`)
- `<NAME>_upper_ui_flutter/`: Flutter 主上位机脚手架, 包含
 - `lib/core/version/`、`lib/app.dart`、`pubspec.yaml`、`version_features.json`, 默认覆盖 App/Web 交付
 - Flutter 侧通过 `String.fromEnvironment('UPPER_VERSION')` / `String.fromEnvironment('UPPER_FEATURES')` 读取编译期常量
 - 生成后会执行 `flutter pub get`, 本地验证至少运行 `flutter analyze` 与 `flutter test`
- 仅当传入 `--with-mini-program` 时, 额外生成 `<NAME>_upper_ui/`: Vue3 + Vite + uni-app + Capacitor 小程序/历史兼容栈
 - `vite.config.js` 会读取 `UPPER_VERSION` / `UPPER_FEATURES` 环境变量并注入常量
 - 默认 `version_features.json` 示例中 `versions.default` 仅启用 `common` 模块, 可按小程序/历史版本维护功能差异
 - 生成阶段会执行 `npm install`、`npm run build:h5`、`npm run build:mp-weixin`、`npx cap init`、`npx cap add android`

构建:

```
cd projects/RTJK_001
cmake -S . -B build -G Ninja \
  -DCMAKE_TOOLCHAIN_FILE=../../cmake/stm32-gcc-toolchain.cmake
cmake --build build
```

按版本导出独立工程 (交付、归档、CI 产物) :

```
cd project_template
python tools/export_project.py --project RTJK_001 --batch --
versions 1-10 -o ../exports --clean \
  --extras readme.txt,PRODUCT_SPEC.md
```

导出目录结构:

```
exports/
├── RTJK_001/
│   ├── RTJK_001_version1/          # 裸嵌入式工程 (cmake、app、
│   ├── RTJK_001_version2/          drivers、bsp、common)
│   └── ...
```

```
|   └─ upper_ui/
|       └─ RTJK_001_upper_ui_flutter/
|           └─ version_features.json
|               └─ versions/
|                   └─ version1/      # Flutter 源码 +
build_outputs/flutter-apk
|                   └─ ...
|               └─ RTJK_001_upper_ui/  # 仅项目包含 uni-app 小程序/历史栈时
存在
|                   └─ version_features.json
|                   └─ versions/
|                       └─ version3/    # 小程序源码 + dist/build/mp-
weixin
```

`export_project.py` 在导出每个版本时，会：

- 1. Flutter 主上位机：解析 `<NAME>_upper_ui_flutter/version_features.json` → 获取 `features` → 以 `--dart-define=UPPER_VERSION=<N> / --dart-define=UPPER_FEATURES=<list>` 执行 `flutter build apk --debug`，包含 `web` feature 时同时执行 `flutter build web`，再将 Flutter 工程源码与 `build_outputs/` 拷贝到 `versions/<label>/`。
- 2. uni-app 小程序/历史栈：解析 `<NAME>_upper_ui/version_features.json` → 获取 `features` → 计算需要复制的源码路径（支持 glob / `!` 排除），仅对包含 `mpweixin` 的版本运行 `npm run build:mp-weixin` 并导出到 `versions/<label>/`。
- 3. 双栈项目使用两套独立版本矩阵；Flutter 只导出 Flutter 矩阵显式覆盖的主上位机版本，uni-app 只导出小程序版本。

如未提供 `version_features.json`，工具会退化为兼容导出策略；新项目应始终提供 Flutter 侧版本矩阵。

导出约定与禁止行为见 [512](#)；修改 CMake / 驱动目录结构 / 上位机 feature 定义后应重新导出并编译验证。

可选 HAL 开关：

```
cmake -S . -B build -DHAL_I2C_USE_SOFT=ON -DHAL_SPI_USE_SOFT=ON
```

5.3 版本间差异配置方式

需求	正确做法
A 版有 DHT11，B 版有 DS18B20	两项目 <code>DRIVER_SRCS</code> 各选不同 sensor 源文件
引脚不同	各项目独立 <code>board_config.h</code>

需求	正确做法
业务逻辑不同	各项目独立 <code>app_main.c</code> / <code>app_callbacks.c</code>
应用事件不同	各项目 <code>version_config.h</code> 定义 <code>APP_EVENT_*</code>
功能宏文档化	<code>version_config.h</code> 中强制输出 <code>VERSION_FEATURE_*</code> ；应用/板级代码只消费这些功能宏

5.4 从 ZNCZ_001 复制出新版本

1. 复制 `projects/ZNCZ_001` → `projects/NEW_XXX`。
2. 修改 `CMakeLists.txt` 中 `project(NEW_XXX)` 与 `DRIVER_SRCS`。
3. 修改 `version_config.h` 中 `VERSION_NAME`。
4. 按新硬件改 `board_config.h`。
5. 重写 `app_main.c` 业务，保留启动顺序。

6. 扩展接口类型（新设备类别）

当现有接口类型无法覆盖新硬件时：

1. 在 `common/interfaces/` 新增 `xxx_if.h`，定义函数指针结构体。
2. 在 `driver_core.h` 增加 `DRIVER_TYPE_XXX` 与 `REGISTER_DRIVER` 映射。
3. 在 `devmgr.c` / `devmgr.h` 增加 `init` 分支与 `devmgr_get_xxx()` — **这是唯一允许修改 devmgr 的场景。**
4. 添加示例驱动并更新文档。

禁止为单一驱动在 `devmgr` 里写特殊 case；新类型应对所有同类设备通用。

7. 禁止行为（Anti-Patterns）

以下行为会导致架构退化，**扩展时严禁出现**：

7.1 应用层

禁止	原因	正确做法
<code>#include "dht11.c"</code> 或包含具体驱动头文件	破坏解耦	只用 <code>devmgr.h</code> + 接口头文件

禁止	原因	正确做法
在 <code>app_main.c</code> 里手动调用 <code>dht11_init()</code>	绕过自注册	依赖 <code>devmgr_init_all()</code>
维护手写驱动数组/初始化表	与链接段机制重复	使用 <code>REGISTER_DRIVER</code>
使用 <code>malloc</code> / <code>free</code>	违背静态分配原则	静态数组、编译期确定大小
在应用任务中长时间 busy-wait	阻塞协作调度	<code>task_block()</code> 或 HAL 超时等待
跳过 <code>bsp_init()</code> 直接用 HAL	SysTick/DWT 未就绪	始终最先调用 <code>bsp_init()</code>
用 <code>sprintf</code> / <code>strcat</code> 手工拼业务 JSON	字符串未统一转义且易溢出	用 cJSON API 构建/解析每个字段
在业务代码中直接判断 <code>APP_VERSION</code> 或旧项目宏	版本规则分散, 导出后难验证	在 <code>version_config.h</code> 输出 <code>VERSION_FEATURE_*</code> , 业务只判断功能宏

7.2 驱动层

禁止	原因	正确做法
在驱动内写死 <code>GPIOB</code> <code>Pin_6</code>	无法跨板复用	读 <code>board_config.h</code>
直接调用 <code>GPIO_SetBits</code> 、 <code>I2C_GenerateSTART</code> 等 SPL API	绕过 HAL, 难以移植	调用 <code>gpio_hal_*</code> / <code>i2c_hal_*</code>
在驱动文件定义 <code>void</code> <code>USART1_IRQHandler</code>	与 <code>irq_handlers.c</code> 冲突	通过 HAL + <code>irq_event</code>
恢复已删除的旧 API 封装	造成双轨维护	使用 <code>*_hal_init(&cfg)</code> 配置结构体
未编译进项目的驱动仍被应用按名调用且不判空	运行时崩溃	<code>devmgr_get_*</code> () 后检查 <code>!= 0</code>

禁止	原因	正确做法
驱动间互相 <code>#include</code> 或调用	耦合	通过应用层或共享 HAL 协调

7.3 BSP / 构建

禁止	原因	正确做法
修改链接脚本删除 <code>.driver_list</code>	自注册失效	保留 <code>KEEP(*(.driver_list))</code>
在 <code>common/</code> 里 <code>#include "board_config.h"</code>	公共层依赖具体项目	仅驱动和 app 包含 <code>board_config</code>
为省 Flash 在 <code>core</code> 里 <code>#ifdef VERSION_XXX</code>	公共代码分叉	版本差异放在 <code>projects/</code>
软 I2C 开关打开但 <code>board</code> 仍用 <code>AF_OD</code>	引脚模式错误	<code>#if HAL_I2C_USE_SOFT</code> 分支
启用 DMA 但未在 <code>board</code> 定义 DMA 通道	编译或运行错误	同步 <code>HAL_USART_ENABLE_DMA</code> 与 <code>board</code> 宏

7.4 多版本管理

禁止	原因	正确做法
为每个版本复制整个 <code>common/</code> 或 <code>bsp/</code>	无法维护	只复制 <code>projects/<VERSION>/</code>
在 <code>drivers/</code> 里 <code>#ifdef ZNCZ_001</code>	驱动库被污染	CMake 选择是否编译该文件
同一逻辑名注册两个驱动	devmgr 查询不确定	保证 <code>"name"</code> 全局唯一
新增 <code>XXX_VERSION / XXX_HAS_*</code> 等项目专属主宏	版本入口不统一	统一使用 <code>APP_VERSION</code> → <code>VERSION_FEATURE_*</code>

7.5 导出工具 (export_project.py)

与 §12 重复或冲突的行为一律禁止；常见项：

禁止	原因
在 DRIVER_SRCS 中手写驱动 .c 路径	导出无法复用 driver_catalog.cmake 版本规则
用 option() 定义数值型版本号	CLion 会解析成 BOOL，导出/编译版本错误
把 include(hal_options.cmake) 写在版本相关 if() 之前	preamble 截断失效，HAL 源与版本不匹配
在 common/ / drivers/ 里 #ifdef APP_VERSION 或项目版本宏	公共代码被污染；版本差异应只在 catalog + version_config.h
新增项目专属主版本宏或功能宏（如 MY_VERSION / MY_HAS_WIFI）	多项目规则分叉，导出脚本难统一
用手工字符串拼接业务 JSON 字段	字段转义和缓冲区大小不可控
修改 add_executable 目标名（非 \${PROJECT_NAME}.elf）	导出脚本无法解析源组与编译宏
源文件使用本机绝对路径	导出目录不可移植

8. 推荐做法 (Do's)

8.1 驱动实现

- init 中完成 HAL 初始化，检查 hal_status_t，失败时保持安全默认状态。
- 读操作带超时；I2C 异常时调用 i2c_hal_bus_recover()。
- 协议耗时操作放在应用任务或专用 poll 任务中，避免在 init 中阻塞过久。
- 用文件内 static 变量缓存最近一次有效读数，供快速查询。
- 通信协议（AT 命令等）在驱动内部实现状态机，对外仍暴露 comm_driver_t 四个方法。

8.2 应用任务

- 每个任务末尾调用 `task_block(events)`，避免占满 CPU。
- 传感器/通信轮询：用 `sched_loop` 注册周期任务（见 `RTJK_001` / `ZNCZ_001` 的 `app_main.c`），或手写 `driver_task_t` + `sched_add_task`。
- 复杂模式切换：`ZNCZ_001` 已用 `app_fsm` 管理 UI 状态；其它工程可复用或继续在 `app_logic.c` 手写分支。
- 使用 `version_config.h` 中的 `APP_EVENT_*`，不要与 `SCHED_EVENT_TICK` 冲突。

8.3 调试与发布

- 构建后检查 `.map`：Flash/RAM 占用、`.driver_list` 条目数量。
- 新增 IRQ 时确认 `stm32f10x_it.c` 未重复定义（本项目 IRQ 在 `irq_handlers.c`）。
- 发布前确认 `DRIVER_SRCS` 仅含本版本需要的驱动，避免多余 Flash 占用。

9. 常见维护场景速查

场景	改哪些文件
换 OLED I2C 引脚	<code>projects/*/app/board_config.h</code>
新增 BH1750 光照传感器	<code>drivers/sensors/bh1750.c</code> 、 <code>board_config</code> 、 <code>driver_catalog.cmake</code> 中 <code>DRIVER_CATALOG_*</code>
新增多版本成品工程	<code>driver_catalog.cmake</code> 、 <code>version_config.h</code> 、 <code>§12.5 Checklist</code>
交付独立编译包	<code>python tools/export_project.py ...</code> （ <code>§12.6</code> ）+ <code>§10.1 回归</code>
启用软件 I2C	<code>cmake -DHAL_I2C_USE_SOFT=ON</code> + <code>board_config</code> 引脚模式
ESP8266 改 USART1→USART3	<code>board_config</code> 中 <code>BOARD_ESP01S_USART</code> 、引脚、 <code>Remap</code>
新增应用任务	<code>app_main.c</code> 定义 <code>driver_task_t</code> + <code>sched_add_task</code>
USART 收数据唤醒应用	<code>irq_event_bind</code> + 驱动 <code>usart_hal_enable_rx_irq</code>

场景	改哪些文件
新 MCU F103CB (128K)	换 linker script, 其余 HAL 通常不变
新驱动类型 (如电机)	<code>motor_if.h</code> 、 <code>driver_core</code> 、 <code>devmgr</code> 、示例驱动

10. 文档与代码同步

修改以下内容时, 请同步更新文档:

变更	更新
新增 HAL 模块或 CMake option	architecture.md 、本指南 §2.7 / §4.3
新增接口类型	architecture.md 、本指南 §6
修改启动流程 / sched_loop	architecture.md 、本指南 §2.4 / §8.2
参考工程或构建指标变更	project_status.md
gen_project 模板变更	本指南 §5.2
export_project / driver_catalog / hal_options 变更	本指南 §12、§10.1
新增 DRIVER_CATALOG_* 条目	<code>cmake/driver_catalog.cmake</code> 、本指南 §12
新产品规格	<code>projects/<NAME>/PRODUCT_SPEC.md</code> 、本指南 §11.4
上位机 feature / version_features.json / Flutter dart-define / uni-app vite 常量	upper_ui_guide.md

10.1 导出回归验证 (必做)

修改 §12 涉及的路径、CMake 结构、`hal_options.cmake` 或 `driver_catalog.cmake` 后, 至少执行:

```
cd project_template
python tools/export_project.py --project RTJK_001 --batch --
versions 1-10 -o ../exports --clean \
    --extras readme.txt,PRODUCT_SPEC.md
python tools/export_project.py --project ZNCZ_001 -o ../exports --
clean --extras readme.txt
```

对每个 exports/<工程>/ 目录: `cmake -S . -B build -G Ninja -`
`DCMAKE_TOOLCHAIN_FILE=cmake/stm32-gcc-toolchain.cmake && cmake --build`
`build`。

11. 当前已知限制（扩展时注意）

与代码同步基准: `RTJK_001` (10 版本)、`ZNCZ_001`、
`driver_catalog.cmake`、`hal_options.cmake`、`export_project.py`。更
细的完成度指标见 [project status.md](#)。

11.1 框架与调度

项	现状	扩展时注意
驱动接口	各类 <code>*_if.h</code> 仅提供 <code>init</code> + 读写方法, 无框架级 <code>loop()</code>	轮询、协议状态机放在应用任务或驱动内部 static 状态
任务轮询	<code>sched_loop</code> (<code>common/scheduler/sched_loop.*</code>) 已用于 <code>RTJK_001</code> 、 <code>ZNCZ_001</code>	新工程优先用 <code>SCHED_LOOP_DEF</code> + <code>sched_loop_register</code> ; 仍可直接写 <code>driver_task_t</code>
应用状态机	<code>app_fsm</code> (<code>common/app_framework/</code>)	ZNCZ_001 已接入 (5 个 UI 状态 + 按键/MQTT 事件) ; <code>RTJK_001</code> 仍用手写 <code>app_logic</code>
中断唤醒	<code>irq_event</code> + <code>task_block(wait_events)</code>	无 IRQ 的设备 (如 NRF24 分包) 须在 <code>comm_poll</code> 等任务中周期性 <code>comm_port_recv()</code>

11.2 HAL 能力与槽位

模块	已实现	限制
I2C	软件 / 硬件二选一 (<code>HAL_I2C_USE_SOFT</code>)	软、硬 I2C 均按 I2C1/I2C2 两路逻辑槽位 ；第 3 路需扩展 <code>i2c_hal_*</code> 数组与 <code>board_config</code>
SPI	软件 (<code>HAL_SPI_USE_SOFT</code>) 与 硬件 (<code>HAL_SPI_USE_HW</code>) 互斥 ； 硬件 SPI 可选 DMA 全双工 TX/RX (<code>HAL_SPI_ENABLE_DMA</code>)	<code>bus_id</code> 上限 <code>SPI_HAL_BUS_MAX</code> (2 ，对应 SPI1/SPI2)；RC522 等需 <code>DHAL_SPI_USE_HW=ON</code>
USART	IRQ 收发 + 可选 DMA TX (<code>HAL_USART_ENABLE_DMA</code>)	RX 无 DMA ，仍为 IRQ + 环形缓冲；DMA 仅走 <code>usart_hal_send_buffer</code> 发送路径
ADC	轮询 (<code>HAL_ADC_ENABLE</code>) + 可选 DMA 连续采集 (<code>HAL_ADC_ENABLE_DMA</code>)	是否启用由工程 CMake preamble 决定 (如 RTJK <code>APP_VERSION>=8</code> 开 ADC)；DMA 与 RTJK MSP20 当前用轮询 ADC
Timer / GPIO	已完成	—
Debug UART	<code>HAL_DEBUG_UART_ENABLE</code> + PC13 软串口 TX	与 LED 等同脚时 LED 驱动自动禁用；高日志量影响实时性

11.3 驱动库 (28 个 .c 源文件)

驱动已统一在 `drivers/`，通过 `driver_catalog.cmake` 按工程/版本选用，并非“多数桩”：

完成度	代表驱动	说明
高	<code>esp8266_wifi</code> 、 <code>esp8266_mqtt</code> 、OLED/按键/继电器、 <code>max30102</code> 、 <code>mcp20_bp</code> (RTJK V8+)	已在参考工程中链接并编译通过
中	<code>jdy31_ble</code> 、 <code>su03t_voice</code> 、 <code>l76k_gnss</code> 、 <code>nrf24l01</code>	透传/NMEA/分包框架就绪，需实板验证

完成度	代表驱动	说明
因器件而异	DHT11、BMP180、MPU6050、HX711、RFID 等	协议代码已迁移，精度/时序需板级实测

新增或完善协议时仍遵循 §3；不要在 `drivers/` 内写 `#ifdef RTJK_001` 等工程名（见 §12.3）。

11.4 参考工程与工具

项	现状
<code>ZNCZ_001</code>	智能插座 Demo：手动/定时、5 键、DS1302、ESP8266 MQTT JSON； <code>sched_loop</code> 四任务
<code>RTJK_001</code>	健康监测： <code>APP_VERSION</code> 1-10， <code>VERSION_FEATURE_*</code> 条件编译 + <code>driver_catalog</code> 选驱动；默认 V10
<code>driver_all_test</code>	全量驱动链接冒烟（约 26 条 <code>.driver_list</code> ）
<code>export_project.py</code>	多版本导出独立工程；约定见 §12；改 CMake/catalog 后须 §10.1 回归
<code>gen_project.py</code>	仅生成最小 <code>app_main</code> / <code>board_config</code> 骨架， 不含 <code>app_logic</code> 、 <code>comm_port</code> 、MQTT 模板

11.5 产品级缺口（非框架缺陷）

项	说明
实板联调	多数驱动未经全量硬件回归
掉电保存	ZNCZ 定时开/关、RTJK 阈值等默认在 RAM，未做 Flash/EEPROM 持久化
ESP8266 断线重连	启动连接为主，无完整自动重连策略
多版本维护成本	RTJK 10 版需同步 <code>version_config.h</code> 、 <code>driver_catalog</code> 、文档；导出后须逐版编译验证

12. 导出工具兼容规范（export_project.py）

tools/export_project.py 将 projects/<NAME>/ 按版本展开为 **独立可编译目录**（默认输出到仓库 exports/）。维护者新增工程、改 CMake、改 HAL/驱动目录时，必须遵守本节，否则导出失败或导出版本与模板内编译不一致。

12.1 工具如何解析工程

步骤	数据来源	说明
源文件组	工程 CMakeLists.txt 中 set(APP_SRCS ...) 等 + add_executable	路径含 \${TEMPLATE_ROOT}/ 或 app/
HAL 源与开关	cmake/hal_options.cmake + preamble	通过 tools/cmake/resolve_export.cmake 脚本模式解析
HAL 前条件	include(hal_options) 之前的 CMake 片段	如 APP_VERSION=>=8 时 set(HAL_ADC_ENABLE ON ...)
驱动列表	cmake/driver_catalog.cmake 中 DRIVER_CATALOG_<NAME>	由 set(DRIVER_SRCS \${DRIVER_CATALOG_*}) 引用
编译宏	target_compile_definitions 静态项 + HAL 宏 + 版本变量	APP_VERSION=<N> 由导出脚本注入；旧项目宏仅兼容
工具链	STM32_TOOLCHAIN_BIN	CLI --toolchain-bin / 环境变量 STM32_TOOLCHAIN_BIN 可覆盖
上位机特性	Flutter 主栈读取 projects/<NAME>/<NAME>_upper_ui_flutter/version_features.json； uni-app 小程序/历史栈读取 projects/<NAME>/<NAME>_upper_ui/version_features.json	defaultFeatures + versions.<N>.features 决定 UPPER_FEATURES；uni-app 的 featureGlobs + alwaysInclude 决定 导出哪些源码

原则：版本差异与选源逻辑写在 CMake + driver_catalog + version_config.h，不要写进 export_project.py。

导出源码裁剪：默认导出会根据当前 APP_VERSION、VERSION_FEATURE_* 与 HAL_*，保守移除一手源码中已确定未启用的条件编译分支。无法安全求值的条件块（如 BOARD_*、include guard、第三方库平台宏）会原样保留。调试或对比原始条件块时可传 --no-prune-conditionals 关闭裁剪。裁剪结果会写入 export_manifest.txt 的 [prune_macros]、[prune_stats]、[prune_warnings]。

12.2 CMake 编写规则（必须遵守）

12.2.1 工程 CMakeLists.txt 骨架

每个 `projects/<NAME>/CMakeLists.txt` 应遵循下列顺序与写法（与 `RTJK_001`、`ZNCZ_001` 一致）：

```
cmake_minimum_required(VERSION 3.20)
set(CMAKE_SYSTEM_NAME Generic)
set(CMAKE_SYSTEM_PROCESSOR arm)
set(CMAKE_TRY_COMPILE_TARGET_TYPE STATIC_LIBRARY)

project(MY_PROJECT C ASM)

set(TEMPLATE_ROOT ${CMAKE_CURRENT_LIST_DIR}/../..)

# ① 版本号、option、与版本相关的 HAL 开关（必须在 hal_options 之前）
set(APP_VERSION 1 CACHE STRING "Application version (1-5)")
if(APP_VERSION LESS 1 OR APP_VERSION GREATER 5)
    message(FATAL_ERROR "APP_VERSION must be 1-5, got
'${APP_VERSION}'")
endif()

# ② 固定写法，路径勿改——export 靠此截取 preamble
include(${TEMPLATE_ROOT}/cmake/hal_options.cmake)
include(${TEMPLATE_ROOT}/cmake/driver_catalog.cmake)

set(STM32_TOOLCHAIN_BIN "... " CACHE PATH "...")
set(LINKER_SCRIPT
${TEMPLATE_ROOT}/bsp/linker_scripts/STM32F103XX_FLASH.ld)
set(STARTUP_FILE
${TEMPLATE_ROOT}/bsp/startup/startup_stm32f103xb.s)

set(CPU_FLAGS
    -mcpu=cortex-m3
    -mthumb
)

set(COMMON_SRCS
    ${TEMPLATE_ROOT}/common/...
)

set(BSP_SRCS
    ${TEMPLATE_ROOT}/bsp/...
    ${BSP_HAL_I2C_SRC}      # HAL 条件源：仅引用 hal_options 产生的变量
    ${BSP_HAL_ADC_SRC}
)

set(SPL_SRCS
    ${TEMPLATE_ROOT}/bsp/stm32f10x_std_lib/...
    ${SPL_HAL_I2C_SRC}
```

```
)
set(DRIVER_SRCS ${DRIVER_CATALOG_MY_PROJECT}) # ③ 必须引用 catalog
变量

set(APP_SRCS
    app/app_main.c
    app/app_logic.c
)

add_executable(${PROJECT_NAME}.elf
    ${APP_SRCS}
    ${COMMON_SRCS}
    ${BSP_SRCS}
    ${DRIVER_SRCS}
    ${SPL_SRCS}
    ${STARTUP_FILE}
)

target_include_directories(${PROJECT_NAME}.elf PRIVATE
    app
    ${TEMPLATE_ROOT}/common/interfaces
    ...
)

target_compile_definitions(${PROJECT_NAME}.elf PRIVATE
    STM32F10X_MD
    USE_STDPERIPH_DRIVER
    APP_VERSION=${APP_VERSION}
)

hal_apply_compile_definitions(${PROJECT_NAME}.elf)
```

12.2.2 源路径与变量

规则	正确示例	错误示例
框架内源文件	<code>\${TEMPLATE_ROOT}/drivers/sensors/dht11.c</code>	<code>D:/dev/PerfabProj/.../dht11.c</code>
应用源文件	<code>app/app_main.c</code> (相对 <code>projects/<NAME>/</code>)	<code>projects/MY/app_main.c</code>

规则	正确示例	错误示例
驱动选用	<code>set(DRIVER_SRCS \${DRIVER_CATALOG_MY_PROJECT})</code>	<code>list(APPEND DRIVER_SRCS \${TEMPLATE_ROOT}/drivers/...)</code>
HAL条件源	<code>\${BSP_HAL_ADC_SRC}、 \${SPL_HAL_DMA_SRC}</code>	在工程 CMake 里手写 <code>adc_hal.c</code>
启动/链接脚本	<code>set(STARTUP_FILE \${TEMPLATE_ROOT}/bsp/startup/...)</code>	省略或使用相对仓库外的路径

12.2.3 多版本工程

规则	说明
版本变量命名	<code>set(APP_VERSION N CACHE STRING "Application version (...)"</code> ; <code>APP_VERSION</code> 是唯一全局版本输入宏, 旧 <code><前缀>VERSION</code> 只作为兼容别名
版本类型	禁止 <code>option(APP_VERSION ...)</code> ; 必须用 <code>CACHE STRING</code>
版本范围	<code>message(FATAL_ERROR "APP_VERSION must be 1-10")</code> 供导出脚本解析批量范围
版本→驱动	在 <code>driver_catalog.cmake</code> 的 <code>DRIVER_CATALOG_<NAME></code> 内用归一化后的 APP 版本局部变量
版本→HAL	在 include(hal_options) 之前的 preamble 中 <code>set(HAL_* ... FORCE)</code>
版本→C宏	<code>version_config.h</code> 根据 <code>APP_VERSION</code> 输出 <code>VERSION_FEATURE_*</code> ; <code>VERSION_FEATURE_*</code> 是唯一功能条件编译宏; 导出时会改写 <code>APP_VERSION</code> 默认值

12.2.4 hal_options.cmake 扩展

新增可选 HAL 模块时：

```
option(HAL_FOO_ENABLE "... " OFF)

if(HAL_FOO_ENABLE)
    list(APPEND BSP_HAL_FOO_SRC
    ${TEMPLATE_ROOT}/bsp/hal_wrapper/foo_hal.c)
endif()
```

- option 必须以 HAL_ 开头，且写在**单独一行**（供 hal_export_utils.cmake 扫描）。
- 条件源必须通过 list(APPEND <大写>_SRC ...) 追加，变量名以 _SRC 结尾。
- 在 hal_apply_compile_definitions() 中同步增加对应 HAL_FOO_ENABLE 宏。

12.2.5 driver_catalog.cmake 扩展

```
set(DRIVER_CATALOG_MY_PROJECT
    ${TEMPLATE_ROOT}/drivers/displays/oled_font.c
    ...
)

if(DRIVER_CATALOG_MY_PROJECT_VERSION GREATER_EQUAL 3)
    list(APPEND DRIVER_CATALOG_MY_PROJECT
    ${TEMPLATE_ROOT}/drivers/comm/esp8266_wifi.c)
endif()
```

- 每条路径必须 \${TEMPLATE_ROOT}/drivers/... 形式。
- catalog 名与工程名对应： DRIVER_CATALOG_<NAME> ↔ set(DRIVER_SRCS \${DRIVER_CATALOG_<NAME>})。
- **禁止**在 Python 或工程 CMakeLists 中复制一份驱动列表。

12.3 代码编写规则（保障导出版本正确）

类别	规则
目录	应用代码仅放在 projects/<NAME>/app/；可被 app/ 前缀解析
板级配置	每工程独立 board_config.h；引脚/外设实例不写死在 drivers/

类别	规则
版本功能宏	在 <code>version_config.h</code> 用 <code>#if APP_VERSION >= N</code> 生成 <code>VERSION_FEATURE_*</code> ；应用层/板级代码只判断 <code>VERSION_FEATURE_*</code> ，不直接判断版本号
头文件依赖	新增驱动所需 <code>.h</code> 应位于 <code>target_include_directories</code> 已列目录下，或驱动同目录 <code>.h</code> （导出会随驱动复制）
条件编译	版本差异优先 <code>#if</code> + catalog 选源； 禁止 在 <code>common/</code> 、 <code>bsp/hal_wrapper/</code> 、 <code>drivers/</code> 源文件中 <code>#ifdef ZNCZ_001 / #ifdef RTJK_001</code>
JSON 协议	业务 JSON 字段必须通过 cJSON API 构建/解析，禁止 <code>sprintf</code> /字符串拼接生成 JSON
未链接驱动	应用层 <code>devmgr_get_*</code> () 必须判空；catalog 未编入的驱动不应被强依赖
上位机特性资源	<code>version_features.json</code> 的 glob 路径应保持可导出：所有条目都以相对路径（相对上位机根目录）书写，并可匹配到真实文件；排除规则使用 <code>!glob</code> 表示

12.4 严禁行为（导出相关）

以下行为会导致 `export_project.py` 失败 或 导出版与模板内编译不一致：

CMake / 构建

严禁	后果
修改 <code>add_executable</code> 目标名（不用 <code>\${PROJECT_NAME}.elf</code> ）	无法解析源组、 <code>target_compile_definitions</code> 、链接选项
将 <code>include(\${TEMPLATE_ROOT}/cmake/hal_options.cmake)</code> 改为其它路径或拆成变量	preamble 截断失败，HAL 源错误
在 <code>include(hal_options)</code> 之后才根据版本打开 <code>HAL_ADC_ENABLE</code> 等	导出版缺少 ADC/SPI/DMA 等源文件
<code>set(DRIVER_SRCS ...)</code> 内直接列 <code>.c</code> 而不经 <code>DRIVER_CATALOG_*</code>	版本批量导出全部相同驱动集
<code>DRIVER_SRCS</code> 一行内混合 catalog 与手写路径且无法识别 catalog	驱动解析失败
源文件列表使用本机绝对路径	导出目录不可移植、CI 失败

严禁	后果
用 <code>add_subdirectory</code> / <code>FetchContent</code> 拉取源而不写入 <code>*_SRCS</code>	导出目录缺文件
把 <code>set(XXX_SRCS ...)</code> 写成单行超长且含未闭合括号	Python 解析 <code>set()</code> 块失败
数值版本用 <code>option()</code> 定义	CLion/导出均可能得到错误版本
在 <code>export_project.py</code> 硬编码某工程的驱动/HAL 规则	破坏通用性；应改 <code>driver_catalog</code> / <code>preamble</code>

hal_options / 导出脚本

严禁	后果
HAL 开关不用 <code>option(HAL_* ...)</code> （例如仅 <code>set(HAL_XXX ON)</code> 无 <code>option</code> ）	扫描器漏掉开关，导出版宏与源不一致
<code>list(APPEND</code> 与变量名分行书写	当前扫描器按单行匹配，可能漏 <code>_SRC</code> 组
修改 <code>tools/cmake/resolve_export.cmake</code> 却不跑 S10.1 回归	批量导出静默错误

C 源码 / 目录

严禁	后果
在 <code>common/</code> 、 <code>drivers/</code> 写 <code>#ifdef <工程名></code>	导出版仍编译进错误分支
在业务代码中直接判断 <code>APP_VERSION</code> 或旧项目版本宏	版本条件分散，功能裁剪不可审计
新增 <code>XXX_VERSION</code> / <code>XXX_HAS_*</code> 作为主宏	破坏统一版本模型
用 <code>sprintf</code> / <code>strcat</code> / 字符串模板拼业务 JSON	JSON 转义和缓冲区安全不可控
应用源放在 <code>projects/<NAME>/</code> 根目录非 <code>app/</code>	路径解析/版本补丁失效
依赖未列入 CMake 的生成代码或外部树	导出目录缺文件
仅存在于 <code>projects/</code> 的头文件却未加入 <code>target_include_directories</code>	导出版编译找不到头文件

12.5 新建工程 Checklist

1. `python tools/gen_project.py <NAME>` 默认生成 Flutter 主上位机；如需微信小程序，再使用 `python tools/gen_project.py <NAME> --with-mini-program` 生成双栈。
2. `cmake/driver_catalog.cmake` 增加 `DRIVER_CATALOG_<NAME>`（含版本条件）。
3. `CMakeLists.txt`： `project(<NAME>)`、`DRIVER_SRCS`、`include(hal_options)` 顺序符合 §12.2。
4. `app/version_config.h`、`app/board_config.h` 就绪；版本输入统一为 `APP_VERSION`，功能输出统一为 `VERSION_FEATURE_*`。
5. 如涉及 MQTT/BLE/串口 JSON 协议，确认字段构建/解析全部使用 cJSON API。
6. Flutter 上位机：维护 `<NAME>_upper_ui_flutter/version_features.json`，并执行 `flutter analyze / flutter test`。
7. 如含微信小程序：维护 `<NAME>_upper_ui/version_features.json` 与 `src/features/`，并验证 `npm run build:mp-weixin`。
8. 模板内编译：`cmake -S projects/<NAME> -B build ... && cmake --build build`。
9. 导出：`python tools/export_project.py --project <NAME> ... --extras readme.txt`。
10. 对 `exports/<NAME>*/` 逐个配置并编译 (§10.1)。

12.6 常用导出命令

```
# 单版本
python tools/export_project.py --project RTJK_001 --version 10 -o
../exports \
    --extras readme.txt,PRODUCT_SPEC.md

# 批量
python tools/export_project.py --project RTJK_001 --batch --
versions 1-10 -o ../exports --clean \
    --extras readme.txt,PRODUCT_SPEC.md

# 指定工具链（或使用环境变量 STM32_TOOLCHAIN_BIN）
python tools/export_project.py --project ZNCZ_001 -o ../exports \
    --toolchain-bin D:/path/to/GNU-tools-for-STM32/bin

# 不复制附加文档
python tools/export_project.py --project ZNCZ_001 --no-extras
```

导出生成: `CMakeLists.txt` (展开路径)、全部 `.c/.s/.h`、`cmake/stm32-gcc-toolchain.cmake`、`export_manifest.txt`; 附加文件由 `--extras` 指定。

最后更新: 补充 cJSON 构建 JSON 字段强制要求、统一 `APP_VERSION` → `VERSION_FEATURE_*` 条件编译宏规范。